PyPTO: Hello, World!

作者：王子楠——PyPTO 仓 Committer

时间：2026/03/09

https://gitcode.com/cann/pypto

CANN

目录

Part 1 PyPTO 框架概述

Part 2 PyPTO 产生背景

Part 3 PyPTO 环境准备

Part 4 PyPTO 你好世界

https://gitcode.com/cann/pypto

CANN

PyPTO(Python Parallel Tensor/Tile Operation)：高性能编程框架

PyPTO 是什么?

- **本质：** 面向 AI 加速器的高性能编程框架
- **目的：** 简化算子开发流程，同时保持高性能计算能力
- **范式：** 创新的 PTO 编程
- **理念：** 基于 Tensor/Tile 的编程模型

PyPTO 的定位

- 不仅限于单个算子的开发，更强调**复杂融合算子，甚至于整个模型网络**的开发能力
- 已有编程框架的**补充而非替换**，特别适合需要深度性能优化和细粒度控制的场景

https://gitcode.com/cann/pypto

PyPTO

- **Tensor/Tile 编程**
  以 Tensor/Tile 为粒度编程
  屏蔽不必要的优化细节
- **Python 原生**
  提供 Python 原生接口
  降低用户开发门槛
- **跨代兼容**
  通过底层 Tile 指令集
  实现跨代兼容
- **白盒化调优**
  充分发挥专家经验
  编译流程白盒化、可视化

CANN

PyPTO：主打易用性，兼顾通用性和性能

Tensor 级别编程提升易用性

降低开发门槛(好开发），简化开发流程(好使用)

- **基于 jit kernel 整网接入：**
  - 简化开发流程
  - 通过 @allow_in_graph 方式去掉算子入图交付件
- **Tensor编程：** Tensor DSL 编程，易用性最佳，兼顾通用性
  - 具有一定的代际兼容性
  - 基于0.5X理论的基础性能

https://gitcode.com/cann/pypto

CANN

本章小结

- **本章对 PyPTO 编程框架进行了概述**
  - PyPTO 是面向 AI 加速器的高性能编程框架
  - 本次系列课程介绍 Tensor 层次编程，强调易用性

https://gitcode.com/cann/pypto

CANN

目录

Part 1 PyPTO 框架概述

Part 2 PyPTO 产生背景

Part 3 PyPTO 环境准备

Part 4 PyPTO 你好世界

PyPTO 产生的背景和动机—— AI 加速器编程的挑战

AI 加速器编程的挑战	当前核心难点
传统算子开发的复杂性	理解算子的数学计算属性 &考虑硬件友好的执行方式
算法开发与算子开发的鸿沟	算法开发专注于算法逻辑和数学表达 &算子开发要将算法映射到硬件执行
硬件多样性与编程抽象	高层抽象难以充分利用硬件特性 &底层抽象显著增加开发复杂度
性能优化与开发效率	性能优化需要深入的专业知识和大量的调优工作 &快速迭代和验证算法需要高效的开发工具

https://gitcode.com/cann/pypto

CANN

PyPTO 解决的核心问题——聚焦易用性，兼顾性能

消除算法与算子开发的鸿沟

- 通过 Tensor 级别抽象，算法开发者可以直接实现高性能算子，无需等待专门的算子开发团队。
- 这消除了传统分工模式带来的沟通成本和迭代延迟。

平衡编程简单性和控制力

- 通过分层抽象设计，对不同开发者暴露不同抽象层次。
- 算法开发者可以使用高层次的 Tensor API，简化开发流程，降低开发门槛。Tensor 级别抽象下，开发者仍可通过 Tile 切分等方式进行性能优化。

支持快速迭代和深度优化

- 提供“快速可用 → 灵活调优 → 深度优化”的平滑过渡。大多数开发者可以快速获得可用的实现。
- 而性能敏感的开发者可以通过工具链进行深度优化，追求极致性能。

提供完整的工具链支持

- 可视化、调试、性能分析一体化支持，帮助开发者快速定位问题、理解性能瓶颈，并实现深度定制
- 可视化工具和分析工具，高质量示例和文档，与主流 IDE 和构建系统的集成，这些都是生态成功的关键因素。

https://gitcode.com/cann/pypto

CANN

本章小结

- **本章阐述了 PyPTO 的产生背景**
  - 当前 AI 加速器编程呈现开发复杂度高、算法开发与算子开发存在鸿沟等问题
  - PyPTO 通过平衡编程简单性和控制力，聚焦易用性的同时兼顾性能

目录

Part 1 PyPTO 框架概述

Part 2 PyPTO 产生背景

Part 3 PyPTO 环境准备

Part 4 PyPTO 你好世界

https://gitcode.com/cann/pypto/tree/master/docs/install

CANN

PyPTO 环境准备：平台支持

PyPTO 支持平台：昇腾 NPU 与 CPU

环境类型	硬件要求	运行模式
真实环境	配备CPU及NPU硬件	支持在NPU上执行计算，也可以通过CPU仿真获取预估性能
仿真环境	仅有CPU硬件	仅支持通过CPU仿真获取预估性能

- NPU: 指昇腾 AI 处理器
- 支持的系统：PyPTO 支持在 OpenEuler、Ubuntu 等主流 Linux 发行版上编译和运行

https://gitcode.com/cann/pypto/tree/master/docs/install

CANN

PyPTO 环境准备：前提条件

Python 基础依赖

- Python：版本 >= 3.9
- PyTorch 及 Ascend Extenstion for PyTorch
  - 根据实际环境的Python版本单独安装
  - 需确保PyTorch、Ascend Extension for PyTorch 与 PyPTO 三者的 Python 版本一致
  - 仿真环境中可跳过 Ascend Extension for PyTorch 的安装，但仍需安装 PyTorch

编译依赖

- 若不需要编译 PyPTO，可跳过本步骤
- 编译工具
- Python 依赖包
- 第三方开源软件源码包

https://gitcode.com/cann/pypto/tree/master/docs/install

CANN

PyPTO 环境准备：编译依赖

编译依赖安装

- 编译工具：
  - cmake >= 3.16.3
  - make
  - g++ >= 7.3.1
- Python 依赖包：

  ```
  cd pypto
  python3 -m pip install -r python/requirements.txt
  ```
- 第三方开源软件源码包
  - JSON for Modern C++：v3.11.3
  - libboundscheck：v.1.1.16

https://gitcode.com/cann/pypto/tree/master/docs/install

CANN

PyPTO 环境准备：软件包安装

真实环境编译必需软件包

- 驱动与固件：
  - 支持 Ascend HDK 25.3.0、Ascend HDK 25.2.0
- 工具包：
  - 需要下载和安装 toolkit 包、ops 包、PTO-inst 包
  - 可使用脚本bash tools/prepare_env.sh --type=cann --device-type=xx 一键安装
  - 工具包需安装在同一路径下
  - 亦可根据实际环境下载对应的安装包

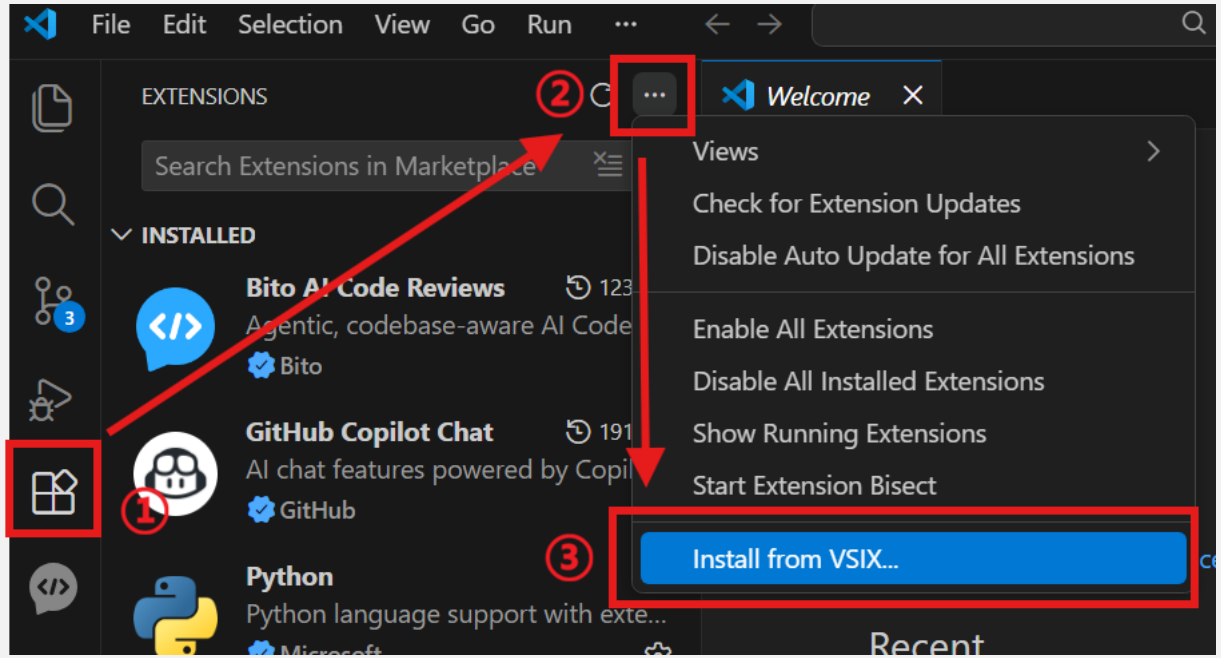
https://gitcode.com/cann/pypto/tree/master/docs/install

CANN

PyPTO 环境准备：PyPTO Toolkit 插件(可选)

PyPTO Toolkit 插件：查看计算图和泳道图

- 下载 .vsix 插件文件
- 打开 Visual Studio Code，进入“扩展”选项卡界面，单机右上角的“...”，选择“从 VSIX 安装...”



- 选择已下载的 .visx 插件文件，完成安装

https://gitcode.com/cann/pypto/tree/master/docs/install

PyPTO 环境准备：编译安装

PyPTO 软件包编译安装

- 通过源码编译安装(推荐)
  - 常规安装：此方式适用于生产环境或代码稳定后使用。编译安装后，对 Python 源码的修改不会体现到已安装的 ‘pypto’ 包中
  - 可编辑安装：此方式适用于开发调试阶段。该模式会在 site-package 目录中创建指向本地源码的软链接，对 Python 源码的修改会即时生效，无需重新安装。
- 通过 PyPI 安装
  - PyPTO 已发布至 PyPI，若不涉及对 PyPTO 源码的修改，可以直接使用 pip 命令安装
- 通过 Docker 镜像安装
  - 为方便快速搭建环境，同样提供已完成 PyPTO 运行环境搭建的 Docker 镜像

https://gitcode.com/cann/pypto/tree/master/docs/install

本章小结

- **本章阐述了 PyPTO 的环境部署与编译安装**
  - 使用 PyPTO 前，确保已安装基础依赖：Python 和 编译依赖；
  - 若需在真实环境中编译运行 PyPTO 并使用其在 NPU 上执行计算的能力，必须安装对应软件包
  - 如需体验计算图和泳道图的查看能力，需安装 PyPTO Toolkit 插件
  - 基础环境搭建完成后，可编译安装 PyPTO 项目，推荐通过源码编译安装

https://gitcode.com/cann/pypto/tree/master/docs/install

目录

Part 1 PyPTO 框架概述

Part 2 PyPTO 产生背景

Part 3 PyPTO 环境准备

Part 4 PyPTO 你好世界

https://gitcode.com/cann/pypto

CANN

PyPTO 样例运行

PyPTO: Hello, World!

- 进入代码目录
  - cd examples/00_hello_world
- 执行用例
  - python3 hello_world.py –run_mode=npu/sim
  - 若是真实可运行环境，run_mode 选用 npu，若是仿真环境，选用 sim
- 用例执行成功

```
Input0 shape: torch.Size([1, 4, 1, 64])
Input1 shape: torch.Size([1, 4, 1, 64])
Output shape: torch.Size([1, 4, 1, 64])
Max difference: 0.000000
✓ Hello world example passed
```

https://gitcode.com/cann/pypto

CANN

PyPTO 样例运行：查看计算图

PyPTO: 查看计算图

- 若需查看计算图，需安装 PyPTO Toolkit 可视化工具

- 在 ${work_path}/output/output_*/ 目录下生成编译和执行的结果文件
  - 右键单机 ${work_path}/output/output_*/program.json 文件，在弹出的菜单中选择“使用 PyPTO Toolkit 打开”

https://gitcode.com/cann/pypto

CANN

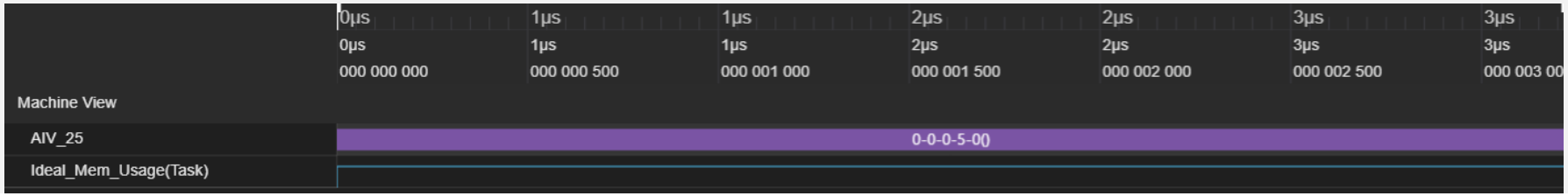
PyPTO 样例运行：查看泳道图

PyPTO: 查看泳道图

- 若需查看泳道图，需安装 PyPTO Toolkit 可视化工具

- 通过 pypto.set_debug_options 接口启动性能数据采集功能
  - 在 hello_world.py Line 57 改为：
  - `return pypto.frontend.jit(debug_options={"runtime_debug_mode": 1})(add_kernel)`

- 重新执行 hello_world.py
  - 在 ${work_path}/output/output_*/ 下生成泳道图数据文件，文件名为：merged_swimlane.json
  - 通过 PyPTO Toolkit 插件查看泳道图



https://gitcode.com/cann/pypto

CANN

本章小结

- **本章阐述了 PyPTO 的 Hello, World! 用例的执行**
  - 根据运行环境选择不同的运行模式
  - 运用 PyPTO Toolkit 进行计算图的查看
  - 运用 PyPTO Toolkit 进行泳道图的查看：通过 pypto.set_debug_options 接口

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann